

جلسه ۷ – مجموعه‌ها (Sets) در پایتون

مقدمه

در این جلسه با **مجموعه‌ها (Set)** در پایتون آشنا می‌شویم. مجموعه یک نوع داده‌ی مهم در پایتون است که برای ذخیره‌سازی چندین مقدار استفاده می‌شود. ویژگی‌های اصلی `set`:

- عناصر مجموعه تکراری نیستند.
- مجموعه‌ها بدون ترتیب (**unordered**) هستند.
- مجموعه‌ها قابل تغییر (**mutable**) هستند، اما عناصر آن باید **immutable** باشند (مثل عدد، رشته یا تاپل).

ایجاد مجموعه (Set)

برای ایجاد یک مجموعه می‌توانیم از آکولاد `{}` یا تابع `set()` استفاده کنیم.

```
In [ ]: # ایجاد مجموعه با {}
numbers = {1, 2, 3, 4, 5}
print(numbers)

# ایجاد مجموعه با تابع set()
letters = set(['a', 'b', 'c', 'a']) # تکراری حذف می‌شود
print(letters)
```

ویژگی‌های مجموعه‌ها

- مجموعه ترتیب خاصی ندارد.
- عناصر تکراری در مجموعه ذخیره نمی‌شوند.
- مجموعه می‌تواند شامل انواع داده‌های مختلف باشد (عدد، رشته، تاپل و ...).

```
In [ ]: mixed_set = {1, "hello", (2,3)}
print(mixed_set)
```

عملیات مجموعه‌ای

مجموعه‌ها امکان انجام عملیات ریاضی روی داده‌ها را فراهم می‌کنند.

- اجتماع (**Union**): `A | B` یا `A.union(B)`
- اشتراک (**Intersection**): `A & B` یا `A.intersection(B)`
- تفاضل (**Difference**): `A - B` یا `A.difference(B)`

• تفاضل متقارن (Symmetric Difference): $A \Delta B$ یا

`A.symmetric_difference(B)`

```
In [ ]: A = {1, 2, 3, 4}
        B = {3, 4, 5, 6}

        print("اجتماع:", A | B)
        print("اشتراک:", A & B)
        print("تفاضل A-B:", A - B)
        print("تفاضل متقارن:", A ^ B)
```

متدهای مهم مجموعه‌ها

- `add(x)`: اضافه کردن یک عنصر
- `remove(x)`: حذف یک عنصر (خطا می‌دهد اگر وجود نداشته باشد)
- `discard(x)`: حذف یک عنصر (بدون خطا)
- `pop()`: حذف یک عنصر تصادفی
- `clear()`: پاک کردن کل مجموعه
- `copy()`: کپی مجموعه

```
In [ ]: s = {1, 2, 3}
        s.add(4)
        print("بعد add:", s)

        s.remove(2)
        print("بعد remove:", s)

        s.discard(10) # خطا نمی‌دهد
        print("بعد discard:", s)

        print("pop:", s.pop())
        print("copy:", s.copy())

        s.clear()
        print("بعد clear:", s)
```

مقایسه مجموعه‌ها

- `issubset()`: بررسی زیرمجموعه بودن
- `issuperset()`: بررسی ابرمجموعه بودن
- `isdisjoint()`: بررسی اشتراک نداشتن

```
In [ ]: A = {1, 2}
        B = {1, 2, 3, 4}
        C = {5, 6}

        print("A زیرمجموعه B است:", A.issubset(B))
```

```
print("B ابرمجموعه A است:", B.issuperset(A))  
print("A و C اشتراک ندارند:", A.isdisjoint(C))
```

مجموعه‌های غیرقابل تغییر (Frozen Set)

گاهی نیاز داریم مجموعه‌ای بسازیم که دیگر تغییر نکند.
برای این کار از `frozenset` استفاده می‌کنیم.

```
In [ ]: fs = frozenset([1, 2, 3, 4])  
print(fs)  
# fs.add(5) # قابل تغییر نیست frozenset خطا می‌دهد چون
```

تمرین: حذف کلمات تکراری از متن ورودی کاربر

یک متن وارد کنید که شامل کلمات تکراری باشد و با استفاده از `set` کلمات تکراری را از آن حذف کنید

```
In [ ]: text = input("یک متن وارد کنید: ")  
words = text.split()  
unique_words = set(words)  
print("کلمات بدون تکرار:", unique_words)
```

تمرین‌ها

1. برنامه‌ای بنویسید که مجموعه‌ای از اعداد را گرفته و تمام مقادیر تکراری را حذف کند.
2. دو مجموعه تعریف کنید و اجتماع، اشتراک، تفاضل و تفاضل متقارن آنها را نمایش دهید.
3. بررسی کنید که آیا یک مجموعه زیرمجموعه مجموعه دیگر است یا خیر.
4. یک مجموعه از رشته‌ها تعریف کنید و همه‌ی آنها را به حروف بزرگ تبدیل کنید.
5. برنامه‌ای بنویسید که یک متن از کاربر گرفته و تمام کلمات یکتا (بدون تکرار) آن را نمایش دهد.
6. یک برنامه بنویسید که بررسی کند آیا دو رشته شامل حروف مشترک هستند یا خیر (با استفاده از مجموعه‌ها).